Chapter 1. Combining Commands

When you work in Windows, macOS, and most other operating systems, you probably spend your time running applications like web browsers, word processors, spreadsheets, and games. A typical application is packed with features: everything that the designers thought their users would need. So, most applications are self-sufficient. They don't rely on other apps. You might copy and paste between applications from time to time, but for the most part, they're separate.

The Linux command line is different. Instead of big applications with tons of features, Linux supplies thousands of small commands with very few features. The command `cat`, for example, prints files on the screen and that's about it. `ls` lists the files in a directory, `mv` renames files, and so on. Each command has a simple, fairly well-defined purpose.

What if you need to do something more complicated? Don't worry. Linux makes it easy to *combine commands* so their individual features work together to accomplish your goal. This way of working yields a very different mindset about computing. Instead of asking "Which app should I launch?" to achieve some result, the question becomes "Which commands should I combine?"

In this chapter, you'll learn how to arrange and run commands in different combinations to do what you need. To keep things simple, I'll introduce just six Linux commands and their most basic uses so you can focus on the more complex and interesting part—combining them—without a huge learning curve. It's a bit like learning to cook with six ingredients, or learning carpentry with just a hammer and a saw. (I'll add more commands to your Linux toolbox in Chapter 5.)

You'll combine commands using *pipes*, a Linux feature that connects the output of one command to the input of another. As I introduce each command (`wc`, `head`, `cut`, `grep`, `sort`, and `uniq`), I'll immediately demonstrate its use with pipes. Some examples will be practical for daily Linux use, while others are just toy examples to demonstrate an important feature.

Input, Output, and Pipes

Most Linux commands read input from the keyboard, write output to the screen, or both. Linux has fancy names for this reading and writing:

stdin (pronounced "standard input" or "standard in")

> The stream of input that Linux reads from your keyboard. When you type any command at a prompt, you're supplying data on stdin.

stdout (pronounced "standard output" or "standard out")

> The stream of output that Linux writes to your display. When you run the `ls` command to print filenames, the results appear on stdout.

Now comes the cool part. You can connect the stdout of one command to the stdin of another, so the first command feeds the second. Let's begin with the familiar `ls -l` command to list a large directory, such as */bin*, in long format:

```
$ ls -l /bin
total 12104
-rwxr-xr-x 1 root root 1113504 Jun  6  2019 bash
-rwxr-xr-x 1 root root  170456 Sep 21  2019 bsd-csh
-rwxr-xr-x 1 root root   34888 Jul  4  2019 bunzip2
-rwxr-xr-x 1 root root 2062296 Sep 18  2020 busybox
-rwxr-xr-x 1 root root   34888 Jul  4  2019 bzcat
⋮
-rwxr-xr-x 1 root root    5047 Apr 27  2017 znew
```

This directory contains far more files than your display has lines, so the output quickly scrolls off-screen. It's a shame that `ls` can't print the information one screenful at a time, pausing until you press a key to continue. But wait: another Linux command has that feature. The `less` command displays a file one screenful at a time:

```
$ less myfile                    View the file; press q to quit
```

You can connect these two commands because `ls` writes to stdout and `less` can read from stdin. Use a pipe to send the output of `ls` to the input of `less`:

```
$ ls -l /bin | less
```

This combined command displays the directory's contents one screenful at a time. The vertical bar (`|`) between the commands is the Linux pipe symbol.[1] It connects the first command's stdout to the next command's stdin. Any command line containing pipes is called a *pipeline*.

Commands generally are not aware that they're part of a pipeline. `ls` believes it's writing to the display, when in fact its output has been redirected to `less`. And `less` believes it's reading from the keyboard when it's actually reading the output of `ls`.

WHAT'S A COMMAND?

The word *command* has three different meanings in Linux, shown in Figure 1-1:

A program

An executable program named and executed by a single word, such as `ls`, or a similar feature built into the shell, such as `cd` (called a *shell builtin*)[2]

A simple command

A program name (or shell builtin) optionally followed by arguments, such as `ls -l /bin`

A combined command

Several simple commands treated as a unit, such as the pipeline `ls -l /bin | less`

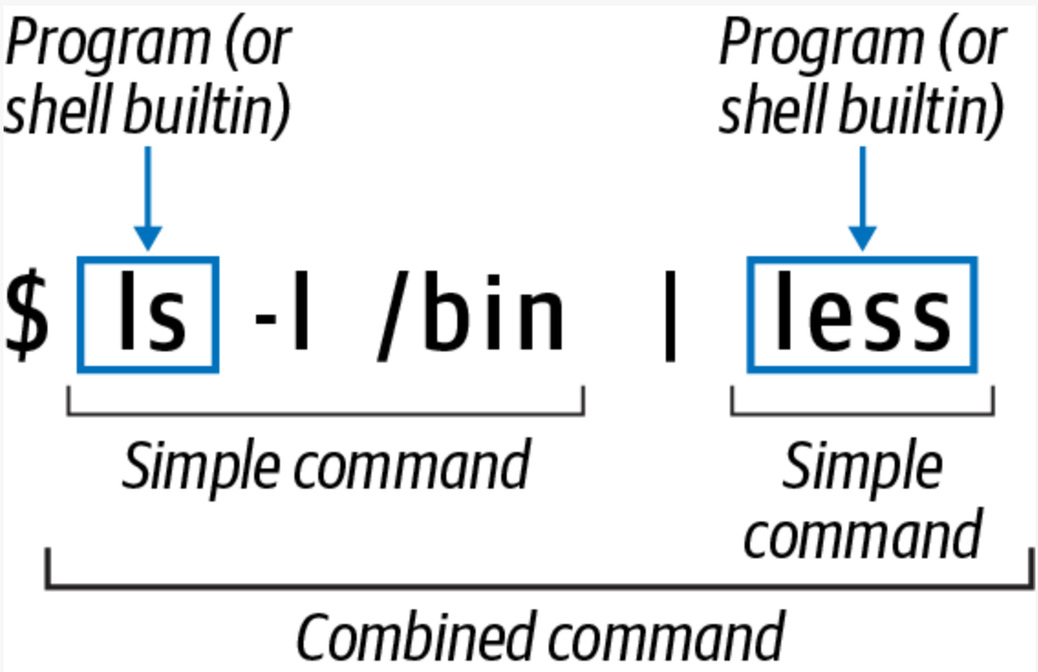

Figure 1-1. Programs, simple commands, and combined commands are all referred to as "commands"

In this book, I'll use the word *command* in all these ways. Usually the surrounding context will make clear which one I mean, but if not, I'll use one of the more specific terms.

Six Commands to Get You Started

Pipes are an essential part of Linux expertise. Let's dive into building your piping skills with a small set of Linux commands so no matter which ones you encounter later, you're ready to combine them.

The six commands—`wc`, `head`, `cut`, `grep`, `sort`, and `uniq`—have numerous options and modes of operation that I'll largely skip for now to focus on pipes. To learn more about any command, run the `man` command to display full documentation. For example:

```
$ man wc
```

To demonstrate our six commands in action, I'll use a file named *animals.txt* that lists some O'Reilly book information, shown in Example 1-1.

Example 1-1. Inside the file animals.txt

```
python  Programming Python      2010    Lutz, Mark
snail   SSH, The Secure Shell   2005    Barrett, Daniel
alpaca  Intermediate Perl       2012    Schwartz, Randal
robin   MySQL High Availability 2014    Bell, Charles
horse   Linux in a Nutshell     2009    Siever, Ellen
donkey  Cisco IOS in a Nutshell 2005    Boney, James
oryx    Writing Word Macros     1999    Roman, Steven
```

Each line contains four facts about an O'Reilly book, separated by a single tab character: the animal on the front cover, the book title, the year of publication, and the name of the first author.

Command #1: wc

The `wc` command prints the number of lines, words, and characters in a file:

```
$ wc animals.txt
  7  51 325 animals.txt
```

`wc` reports that the file *animals.txt* has 7 lines, 51 words, and 325 characters. If you count the characters by eye, including spaces and tabs, you'll find only 318 characters, but `wc` also includes the invisible newline character that ends each line.

The options `-l`, `-w`, and `-c` instruct `wc` to print only the number of lines, words, and characters, respectively:

```
$ wc -l animals.txt
7 animals.txt
$ wc -w animals.txt
51 animals.txt
$ wc -c animals.txt
325 animals.txt
```

Counting is such a useful, general-purpose task that the authors of `wc` designed the command to work with pipes. It reads from stdin if you omit the filename, and it writes to stdout. Let's use `ls` to list the contents of the current directory and pipe them to `wc` to count lines. This pipeline answers the question, "How many files are visible in my current directory?"

```
$ ls -1
animals.txt
myfile
myfile2
test.py
$ ls -1 | wc -l
4
```

The option `-1`, which tells `ls` to print its results in a single column, is not strictly necessary here. To learn why I used it, see the sidebar "ls Changes Its Behavior When Redirected".

`wc` is the first command you've seen in this chapter, so you're a bit limited in what you can do with pipes. Just for fun, pipe the output of `wc` to itself, demonstrating that the same command can appear more than once in a pipeline. This combined command reports that the number of words in the output of `wc` is four: three integers and a filename:

```
$ wc animals.txt
  7  51 325 animals.txt
$ wc animals.txt | wc -w
4
```

Why stop there? Add a third `wc` to the pipeline and count lines, words, and characters in the output "4":

```
$ wc animals.txt | wc -w | wc
      1       1       2
```

The output indicates one line (containing the number 4), one word (the number 4 itself), and two characters. Why two? Because the line "4" ends with an invisible newline character.

That's enough silly pipelines with `wc`. As you gain more commands, the pipelines will become more practical.

LS CHANGES ITS BEHAVIOR WHEN REDIRECTED

Unlike virtually every other Linux command, `ls` is aware of whether stdout is the screen or whether it's been redirected (to a pipe or otherwise). The reason is user-friendliness. When stdout is the screen, `ls` arranges its output in multiple columns for convenient reading:

```
$ ls /bin
bash        dir         kmod      networkctl       red     tar
bsd-csh     dmesg       less      nisdomainname    rm      tempfile
⋮
```

When stdout is redirected, however, `ls` produces a single column. I'll demonstrate this by piping the output of `ls` to a command that simply reproduces its input, such as `cat`:[3]

```
$ ls /bin | cat
bash
bsd-csh
bunzip2
busybox
⋮
```

This behavior can lead to strange-looking results, as in the following example:

```
$ ls
animals.txt   myfile   myfile2    test.py
$ ls | wc -l
4
```

The first `ls` command prints all filenames on one line, but the second command reports that `ls` produced four lines. If you aren't aware of the quirky behavior of `ls`, you might find this discrepancy confusing.

`ls` has options to override its default behavior. Force `ls` to print a single column with the `-1` option, or force multiple columns with the `-C` option.

Command #2: head

The `head` command prints the first lines of a file. Print the first three lines of *animals.txt* with `head` using the option `-n`:

```
$ head -n3 animals.txt
python  Programming Python      2010    Lutz, Mark
snail   SSH, The Secure Shell   2005    Barrett, Daniel
alpaca  Intermediate Perl       2012    Schwartz, Randal
```

If you request more lines than the file contains, `head` prints the whole file (like `cat` does). If you omit the `-n` option, `head` defaults to 10 lines (`-n10`).

By itself, `head` is handy for peeking at the top of a file when you don't care about the rest of the contents. It's a speedy and efficient command, even for very large files, because it needn't read the whole file. In addition, `head` writes to stdout, making it useful in pipelines. Count the number of words in the first three lines of *animals.txt*:

```
$ head -n3 animals.txt | wc -w
20
```

`head` can also read from stdin for more pipeline fun. A common use is to reduce the output from another command when you don't care to see all of it, like a long directory listing. For example, list the first five filenames in the */bin* directory:

```
$ ls /bin | head -n5
bash
bsd-csh
bunzip2
busybox
bzcat
```

Command #3: cut

The `cut` command prints one or more columns from a file. For example, print all book titles from *animals.txt*, which appear in the second column:

```
$ cut -f2 animals.txt
Programming Python
SSH, The Secure Shell
Intermediate Perl
MySQL High Availability
Linux in a Nutshell
```

```
Cisco IOS in a Nutshell
Writing Word Macros
```

`cut` provides two ways to define what a "column" is. The first is to cut by field (`-f`), when the input consists of strings (fields) each separated by a single tab character. Conveniently, that is exactly the format of the file *animals.txt*. The preceding `cut` command prints the second field of each line, thanks to the option `-f2`.

To shorten the output, pipe it to `head` to print only the first three lines:

```
$ cut -f2 animals.txt | head -n3
Programming Python
SSH, The Secure Shell
Intermediate Perl
```

You can also cut multiple fields, either by separating their field numbers with commas:

```
$ cut -f1,3 animals.txt | head -n3
python  2010
snail   2005
alpaca  2012
```

or by numeric range:

```
$ cut -f2-4 animals.txt | head -n3
Programming Python      2010    Lutz, Mark
SSH, The Secure Shell   2005    Barrett, Daniel
Intermediate Perl       2012    Schwartz, Randal
```

The second way to define a "column" for `cut` is by character position, using the `-c` option. Print the first three characters from each line of the file, which you can specify either with commas (`1,2,3`) or as a range (`1-3`):

```
$ cut -c1-3 animals.txt
pyt
sna
alp
rob
hor
don
ory
```

Now that you've seen the basic functionality, try something more practical with `cut` and pipes. Imagine that the *animals.txt* file is thousands of lines long, and you need to extract just the authors' last names. First, isolate the fourth field, author name:

```
$ cut -f4 animals.txt
Lutz, Mark
Barrett, Daniel
Schwartz, Randal
⋮
```

Then pipe the results to `cut` again, using the option `-d` (meaning "delimiter") to change the separator character to a comma instead of a tab, to isolate the authors' last names:

```
$ cut -f4 animals.txt | cut -d, -f1
Lutz
Barrett
Schwartz
⋮
```

> **SAVE TIME WITH COMMAND HISTORY AND EDITING**
>
> Are you retyping a lot of commands? Press the up arrow key instead, repeatedly, to scroll through commands you've run before. (This shell feature is called *command history*.) When you reach the desired command, press Enter to run it immediately, or edit it first using the left and right arrow keys to position the cursor and the Backspace key to delete. (This feature is *command-line editing*.)
>
> I'll discuss much more powerful features for command history and editing in Chapter 3.

Command #4: grep

`grep` is an extremely powerful command, but for now I'll hide most of its capabilities and say it prints lines that match a given string. (More detail will come in Chapter 5.) For example, the following command displays lines from *animals.txt* that contain the string `Nutshell`:

```
$ grep Nutshell animals.txt
horse   Linux in a Nutshell     2009    Siever, Ellen
donkey  Cisco IOS in a Nutshell         2005    Boney, James
```

You can also print lines that *don't* match a given string, with the `-v` option. Notice the lines containing "Nutshell" are absent:

```
$ grep -v Nutshell animals.txt
python  Programming Python      2010    Lutz, Mark
snail   SSH, The Secure Shell   2005    Barrett, Daniel
alpaca  Intermediate Perl       2012    Schwartz, Randal
robin   MySQL High Availability 2014    Bell, Charles
oryx    Writing Word Macros     1999    Roman, Steven
```

In general, `grep` is useful for finding text in a collection of files. The following command prints lines that contain the string `Perl` in files with names ending in *.txt*:

```
$ grep Perl *.txt
animals.txt:alpaca      Intermediate Perl       2012    Schwartz, Randal
essay.txt:really love the Perl programming language, which is
essay.txt:languages such as Perl, Python, PHP, and Ruby
```

In this case, `grep` found three matching lines, one in *animals.txt* and two in *essay.txt*.

`grep` reads stdin and writes stdout, making it great for pipelines. Suppose you want to know how many subdirectories are in the large directory */usr/lib*. There is no single Linux command to provide that answer, so construct a pipeline. Begin with the `ls -l` command:

```
$ ls -l /usr/lib
drwxrwxr-x  12 root root    4096 Mar  1  2020 4kstogram
drwxr-xr-x   3 root root    4096 Nov 30  2020 GraphicsMagick-1.4
drwxr-xr-x   4 root root    4096 Mar 19  2020 NetworkManager
-rw-r--r--   1 root root   35568 Dec  1  2017 attica_kde.so
-rwxr-xr-x   1 root root     684 May  5  2018 cnf-update-db
⋮
```

Notice that `ls -l` marks directories with a `d` at the beginning of the line. Use `cut` to isolate the first column, which may or may not be a `d`:

```
$ ls -l /usr/lib | cut -c1
d
d
d
-
-
⋮
```

Then use `grep` to keep only the lines containing `d`:

```
$ ls -l /usr/lib | cut -c1 | grep d
d
d
d
⋮
```

Finally, count lines with `wc`, and you have your answer, produced by a four-command pipeline—*/usr/lib* contains 145 subdirectories:

```
$ ls -l /usr/lib | cut -c1 | grep d | wc -l
145
```

Command #5: sort

The `sort` command reorders the lines of a file into ascending order (the default):

```
$ sort animals.txt
alpaca  Intermediate Perl       2012    Schwartz, Randal
donkey  Cisco IOS in a Nutshell 2005    Boney, James
horse   Linux in a Nutshell     2009    Siever, Ellen
oryx    Writing Word Macros     1999    Roman, Steven
python  Programming Python      2010    Lutz, Mark
robin   MySQL High Availability 2014    Bell, Charles
snail   SSH, The Secure Shell   2005    Barrett, Daniel
```

or descending order (with the `-r` option):

```
$ sort -r animals.txt
snail   SSH, The Secure Shell   2005    Barrett, Daniel
robin   MySQL High Availability 2014    Bell, Charles
python  Programming Python      2010    Lutz, Mark
oryx    Writing Word Macros     1999    Roman, Steven
horse   Linux in a Nutshell     2009    Siever, Ellen
donkey  Cisco IOS in a Nutshell 2005    Boney, James
alpaca  Intermediate Perl       2012    Schwartz, Randal
```

`sort` can order the lines alphabetically (the default) or numerically (with the `-n` option). I'll demonstrate this with pipelines that cut the third field in *animals.txt*, the year of publication:

```
$ cut -f3 animals.txt                       Unsorted
2010
2005
2012
2014
2009
2005
1999
$ cut -f3 animals.txt | sort -n             Ascending
1999
2005
2005
2009
2010
2012
2014
$ cut -f3 animals.txt | sort -nr            Descending
2014
2012
2010
2009
2005
2005
1999
```

To learn the year of the most recent book in *animals.txt*, pipe the output of `sort` to the input of `head` and print just the first line:

```
$ cut -f3 animals.txt | sort -nr | head -n1
2014
```

> **MAXIMUM AND MINIMUM VALUES**
>
> `sort` and `head` are powerful partners when working with numeric data, one value per line. You can print the maximum value by piping the data to:
>
> ```
> ... | sort -nr | head -n1
> ```
>
> and print the minimum value with:
>
> ```
> ... | sort -n | head -n1
> ```

As another example, let's play with the file */etc/passwd*, which lists the users that can run processes on the system.[4] You'll generate a list of all users in alphabetical order. Peeking at the first five lines, you see something like this:

```
$ head -n5 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
smith:x:1000:1000:Aisha Smith,,,:/home/smith:/bin/bash
jones:x:1001:1001:Bilbo Jones,,,:/home/jones:/bin/bash
```

Each line consists of strings separated by colons, and the first string is the username, so you can isolate the usernames with the `cut` command:

```
$ head -n5 /etc/passwd | cut -d: -f1
root
daemon
bin
smith
jones
```

and sort them:

```
$ head -n5 /etc/passwd | cut -d: -f1 | sort
bin
daemon
jones
root
smith
```

To produce the sorted list of all usernames, not just the first five, replace `head` with `cat`:

```
$ cat /etc/passwd | cut -d: -f1 | sort
```

To detect if a given user has an account on your system, match their username with `grep`. Empty output means no account:

```
$ cut -d: -f1 /etc/passwd | grep -w jones
jones
$ cut -d: -f1 /etc/passwd | grep -w rutabaga        (produces no output)
```

The `-w` option instructs `grep` to match full words only, not partial words, in case your system also has a username that contains "jones", such as `sallyjones2`.

Command #6: uniq

The `uniq` command detects repeated, adjacent lines in a file. By default, it removes the repeats. I'll demonstrate this with a simple file containing capital letters:

```
$ cat letters
A
A
A
B
B
A
C
C
C
C
$ uniq letters
A
B
A
C
```

Notice that `uniq` reduced the first three `A` lines to a single `A`, but it left the last `A` in place because it wasn't *adjacent* to the first three.

You can also count occurrences with the `-c` option:

```
$ uniq -c letters
      3 A
      2 B
      1 A
      4 C
```

I'll admit, when I first encountered the `uniq` command, I didn't see much use in it, but it quickly became one of my favorites. Suppose you have a tab-separated file of students' final grades for a university course, ranging from `A` (best) to `F` (worst):

```
$ cat grades
C       Geraldine
B       Carmine
A       Kayla
A       Sophia
B       Haresh
C       Liam
B       Elijah
B       Emma
A       Olivia
D       Noah
F       Ava
```

You'd like to print the grade with the most occurrences. (If there's a tie, print just one of the winners.) Begin by isolating the grades with `cut` and sorting them:

```
$ cut -f1 grades | sort
A
A
A
B
B
B
B
C
C
D
F
```

Next, use `uniq` to count adjacent lines:

```
$ cut -f1 grades | sort | uniq -c
      3 A
      4 B
      2 C
      1 D
      1 F
```

Then sort the lines in reverse order, numerically, to move the most frequently occurring grade to the top line:

```
$ cut -f1 grades | sort | uniq -c | sort -nr
      4 B
      3 A
      2 C
      1 F
      1 D
```

and keep just the first line with `head`:

```
$ cut -f1 grades | sort | uniq -c | sort -nr | head -n1
      4 B
```

Finally, since you want just the letter grade, not the count, isolate the grade with `cut`:

```
$ cut -f1 grades | sort | uniq -c | sort -nr | head -n1 | cut -c9
B
```

and there's your answer, thanks to a six-command pipeline—our longest yet. This sort of step-by-step pipeline construction is not just an educational exercise. It's how Linux experts actually work. Chapter 8 is devoted to this technique.

Detecting Duplicate Files

Let's combine what you've learned with a larger example. Suppose you're in a directory full of JPEG files and you want to know if any are duplicates:

```
$ ls
image001.jpg  image005.jpg  image009.jpg  image013.jpg  image017.jpg
image002.jpg  image006.jpg  image010.jpg  image014.jpg  image018.jpg
⋮
```

You can answer this question with a pipeline. You'll need another command, `md5sum`, which examines a file's contents and computes a 32-character string called a *checksum*:

```
$ md5sum image001.jpg
146b163929b6533f02e91bdf21cb9563  image001.jpg
```

A given file's checksum, for mathematical reasons, is very, very likely to be unique. If two files have the same checksum, therefore, they are almost certainly duplicates. Here, `md5sum` indicates the first and third files are duplicates:

```
$ md5sum image001.jpg image002.jpg image003.jpg
146b163929b6533f02e91bdf21cb9563  image001.jpg
63da88b3ddde0843c94269638dfa6958  image002.jpg
146b163929b6533f02e91bdf21cb9563  image003.jpg
```

Duplicate checksums are easy to detect by eye when there are only three files, but what if you have three thousand? It's pipes to the rescue. Compute all the checksums, use `cut` to isolate the first 32 characters of each line, and sort the lines to make any duplicates adjacent:

```
$ md5sum *.jpg | cut -c1-32 | sort
1258012d57050ef6005739d0e6f6a257
146b163929b6533f02e91bdf21cb9563
146b163929b6533f02e91bdf21cb9563
17f339ed03733f402f74cf386209aeb3
⋮
```

Now add `uniq` to count repeated lines:

```
$ md5sum *.jpg | cut -c1-32 | sort | uniq -c
      1 1258012d57050ef6005739d0e6f6a257
      2 146b163929b6533f02e91bdf21cb9563
      1 17f339ed03733f402f74cf386209aeb3
      ⋮
```

If there are no duplicates, all of the counts produced by `uniq` will be 1. Sort the results numerically from high to low, and any counts greater than 1 will appear at the top of the output:

```
$ md5sum *.jpg | cut -c1-32 | sort | uniq -c | sort -nr
      3 f6464ed766daca87ba407aede21c8fcc
      2 c7978522c58425f6af3f095ef1de1cd5
      2 146b163929b6533f02e91bdf21cb9563
      1 d8ad913044a51408ec1ed8a204ea9502
      ⋮
```

Now let's remove the nonduplicates. Their checksums are preceded by six spaces, the number one, and a single space. We'll use `grep -v` to remove these lines:[5]

```
$ md5sum *.jpg | cut -c1-32 | sort | uniq -c | sort -nr | grep -v "      1 "
      3 f6464ed766daca87ba407aede21c8fcc
      2 c7978522c58425f6af3f095ef1de1cd5
      2 146b163929b6533f02e91bdf21cb9563
```

Finally, you have your list of duplicate checksums, sorted by the number of occurrences, produced by a beautiful six-command pipeline. If it produces no output, there are no duplicate files.

This command would be even more useful if it displayed the filenames of the duplicates, but that operation requires features we haven't discussed yet. (You'll learn them in "Improving the duplicate file detector".) For now, identify the files having a given checksum by searching with `grep`:

```
$ md5sum *.jpg | grep 146b163929b6533f02e91bdf21cb9563
146b163929b6533f02e91bdf21cb9563  image001.jpg
146b163929b6533f02e91bdf21cb9563  image003.jpg
```

and cleaning up the output with `cut`:

```
$ md5sum *.jpg | grep 146b163929b6533f02e91bdf21cb9563 | cut -c35-
image001.jpg
image003.jpg
```

Summary

You've now seen the power of stdin, stdout, and pipes. They turn a small handful of commands into a collection of composable tools, proving that the whole is greater than the sum of the parts. *Any* command that reads stdin or writes stdout can participate in pipelines.[6] As you learn more commands, you can apply the general concepts from this chapter to forge your own powerful combinations.

---

1 On US keyboards, the pipe symbol is on the same key as the backslash (\), usually located between the Enter and Backspace keys or between the left Shift key and Z.

2 The POSIX standard calls this form of command a *utility*.

3 Depending on your setup, `ls` may also use other formatting features, such as color, when printing to the screen but not when redirected.

4 Some Linux systems store the user information elsewhere.

5 Technically, you don't need the final `sort -nr` in this pipeline to isolate duplicates because `grep` removes all the nonduplicates.

6 Some commands do not use stdin/stdout and therefore cannot read from pipes or write to pipes. Examples are `mv` and `rm`. Pipelines may incorporate these commands in other ways, however; you'll see examples in Chapter 8.

```
FILES="lizard.txt snake.txt"
for f in $FILES; do
  mv mammals/$f reptiles
done
```

Shortening Commands with Aliases

A variable is a name that stands in for a value. The shell also has names that stand in for commands. They're called *aliases*. Define an alias by inventing a name and following it with a equals sign and a command:

```
$ alias g=grep                A command with no arguments
$ alias ll="ls -l"            A command with arguments: quotes are required
```

Run an alias by typing its name as a command. When aliases are shorter than the commands they invoke, you save typing time:

```
$ ll                                        Runs "ls -l"
-rw-r--r-- 1 smith smith 325 Jul  3 17:44 animals.txt
$ g Nutshell animals.txt                     Runs "grep Nutshell animals.txt"
horse   Linux in a Nutshell     2009    Siever, Ellen
donkey  Cisco IOS in a Nutshell 2005    Boney, James
```

> **TIP**
>
> Always define an alias on its own line, not as part of a combined command. (See `man bash` for the technical details.)

You can define an alias that has the same name as an existing command, effectively replacing that command in your shell. This practice is called *shadowing* the command. Suppose you like the `less` command for reading files, but you want it to clear the screen before displaying each page. This feature is enabled with the `-c` option, so define an alias called "less" that runs `less -c`:[2]

```
$ alias less="less -c"
```

Aliases take precedence over commands of the same name, so you have now shadowed the `less` command in the current shell. I'll explain what *precedence* means in "Search Path and Aliases".

To list a shell's aliases and their values, run `alias` with no arguments:

```
$ alias
alias g='grep'
alias ll='ls -l'
```

To see the value of a single alias, run `alias` followed by its name:

```
$ alias g
alias g='grep'
```

To delete an alias from a shell, run `unalias`:

```
$ unalias g
```

Redirecting Input and Output

The shell controls the input and output of the commands it runs. You've already seen one example: pipes, which direct the stdout of one command to the stdin of another. The pipe syntax, `|`, is a feature of the shell.

Another shell feature is redirecting stdout to a file. For example, if you use `grep` to print matching lines from the *animals.txt* file from Example 1-1, the command writes to stdout by default:

```
$ grep Perl animals.txt
alpaca  Intermediate Perl       2012    Schwartz, Randal
```

You can send that output to a file instead, using a shell feature called *output redirection*. Simply add the symbol `>` followed by the name of a file to receive the output:

```
$ grep Perl animals.txt > outfile                    (displays no output)
$ cat outfile
alpaca  Intermediate Perl       2012    Schwartz, Randal
```

You have just redirected stdout to the file *outfile* instead of the display. If the file *outfile* doesn't exist, it's created. If it does exist, redirection overwrites its contents. If you'd rather append to the output file rather than overwrite it, use

the symbol >> instead:

```
$ grep Perl animals.txt > outfile             Create or overwrite outfile
$ echo There was just one match >> outfile     Append to outfile
$ cat outfile
alpaca  Intermediate Perl       2012    Schwartz, Randal
There was just one match
```

Output redirection has a partner, *input redirection*, that redirects stdin to come from a file instead of the keyboard. Use the symbol < followed by a filename to redirect stdin.

Many Linux commands that accept filenames as arguments, and read from those files, also read from stdin when run with no arguments. An example is `wc` for counting lines, words, and characters in a file:

```
$ wc animals.txt                          Reading from a named file
  7  51 325 animals.txt
$ wc < animals.txt                        Reading from redirected stdin
  7  51 325
```

STANDARD ERROR (STDERR) AND REDIRECTION

In your day-to-day Linux use, you may notice that some output cannot be redirected by >, such as certain error messages. For example, ask `cp` to copy a file that doesn't exist, and it produces this error message:

```
$ cp nonexistent.txt file.txt
cp: cannot stat 'nonexistent.txt': No such file or directory
```

If you redirect the output (stdout) of this `cp` command to a file, *errors*, the message still appears on-screen:

```
$ cp nonexistent.txt file.txt > errors
cp: cannot stat 'nonexistent.txt': No such file or directory
```

and the file *errors* is empty:

```
$ cat errors                                  (produces no output)
```

Why does this happen? Linux commands can produce more than one stream of output. In addition to stdout, there is also stderr (pronounced "standard error" or "standard err"), a second stream of output that is traditionally reserved for error messages. The streams stderr and stdout look identical on the display, but internally they are separate. You can redirect stderr with the symbol 2> followed by a filename:

```
$ cp nonexistent.txt file.txt 2> errors
$ cat errors
cp: cannot stat 'nonexistent.txt': No such file or directory
```

and append stderr to a file with 2>> followed by a filename:

```
$ cp nonexistent.txt file.txt 2> errors
$ cp another.txt file.txt 2>> errors
$ cat errors
cp: cannot stat 'nonexistent.txt': No such file or directory
cp: cannot stat 'another.txt': No such file or directory
```

To redirect both stdout and stderr to the same file, use &> followed by a filename:

```
$ echo This file exists > goodfile.txt              Create a file
$ cat goodfile.txt nonexistent.txt &> all.output
$ cat all.output
This file exists
cat: nonexistent.txt: No such file or directory
```

It's *very important* to understand how these two `wc` commands differ in behavior:

- In the first command, `wc` receives the filename *animals.txt* as an argument, so `wc` is aware that the file exists. `wc` deliberately opens the file on disk and reads its contents.
- In the second command, `wc` is invoked with no arguments, so it reads from stdin, which is usually the keyboard. The shell, however, sneakily redirects stdin to come from *animals.txt* instead. `wc` has no idea that the file *animals.txt* exists.

The shell can redirect input and output in the same command:

```
$ wc < animals.txt > count
$ cat count
```

```
      7  51 325
```

and can even use pipes at the same time. Here, `grep` reads from redirected stdin and pipes the results to `wc`, which writes to redirected stdout, producing the file *count*:

```
$ grep Perl < animals.txt | wc > count
$ cat count
      1       6      47
```

You'll dive deeper into such combined commands in Chapter 8 and see many other examples of redirection throughout the book.

Disabling Evaluation with Quotes and Escapes

Normally the shell uses whitespace as a separator between words. The following command has four words—a program name followed by three arguments:

```
$ ls file1 file2 file3
```

Sometimes, however, you need the shell to treat whitespace as significant, not as a separator. A common example is whitespace in a filename such as *Efficient Linux Tips.txt*:

```
$ ls -l
-rw-r--r-- 1 smith smith 36 Aug  9 22:12 Efficient Linux Tips.txt
```

If you refer to such a filename on the command line, your command may fail because the shell treats the space characters as separators:

```
$ cat Efficient Linux Tips.txt
cat: Efficient: No such file or directory
cat: Linux: No such file or directory
cat: Tips.txt: No such file or directory
```

To force the shell to treat spaces as part of a filename, you have three options—single quotes, double quotes, and backslashes:

```
$ cat 'Efficient Linux Tips.txt'
$ cat "Efficient Linux Tips.txt"
$ cat Efficient\ Linux\ Tips.txt
```

Single quotes tell the shell to treat every character in a string literally, even if the character ordinarily has special meaning to the shell, such as spaces and dollar signs:

```
$ echo '$HOME'
$HOME
```

Double quotes tell the shell to treat all characters literally except for certain dollar signs and a few others you'll learn later:

```
$ echo "Notice that $HOME is evaluated"              Double quotes
Notice that /home/smith is evaluated
$ echo 'Notice that $HOME is not'                    Single quotes
Notice that $HOME is not
```

A backslash, also called the *escape character*, tells the shell to treat the next character literally. The following command includes an escaped dollar sign:

```
$ echo \$HOME
$HOME
```

Backslashes act as escape characters even within double quotes:

```
$ echo "The value of \$HOME is $HOME"
The value of $HOME is /home/smith
```

but not within single quotes:

```
$ echo 'The value of \$HOME is $HOME'
The value of \$HOME is $HOME
```

Use the backslash to escape a double quote character within double quotes: